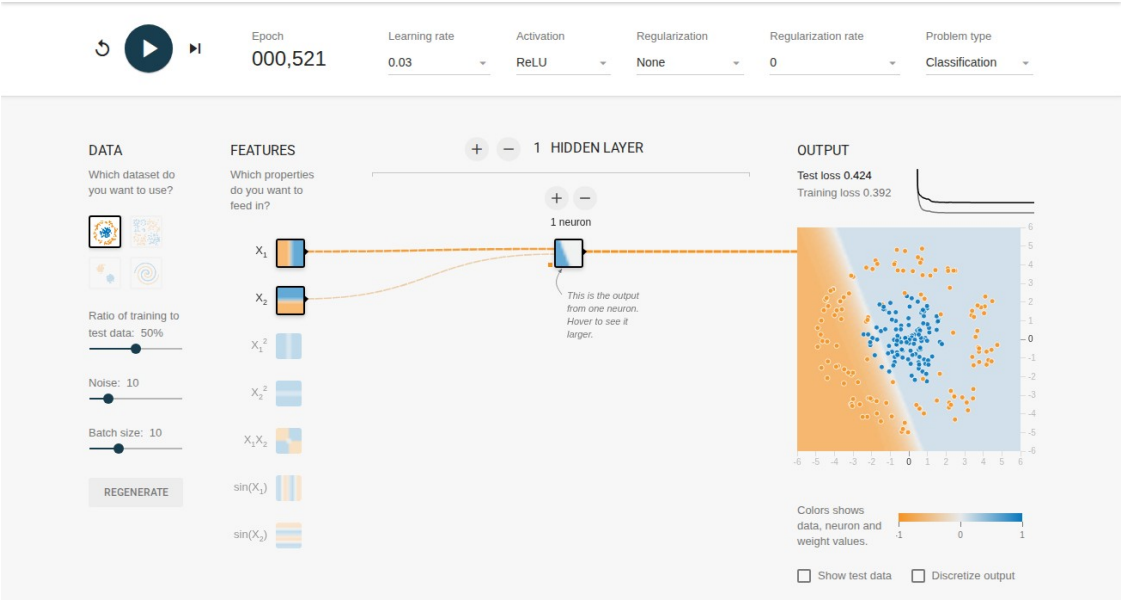
Deep Learning – Lab 4, Part 2

Neural networks on TensorFlow Playground (ENSIA 2026-2027)

Exercise 1 – Learning nonlinearities

Dataset: circle, noise: 10%, features X1 and X2 only.

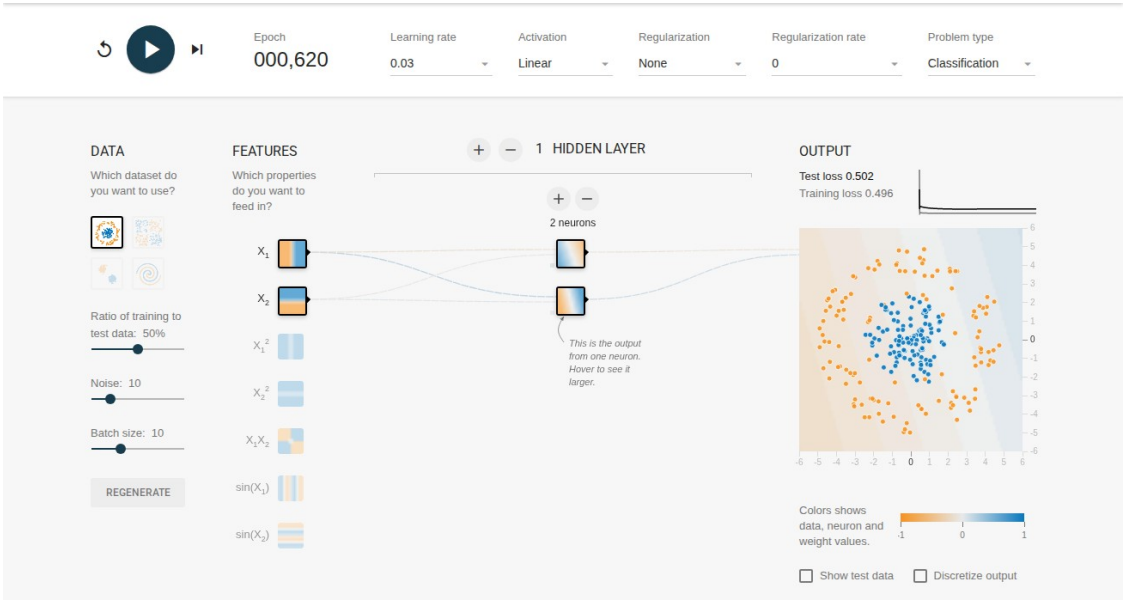
Task 1 – one hidden neuron



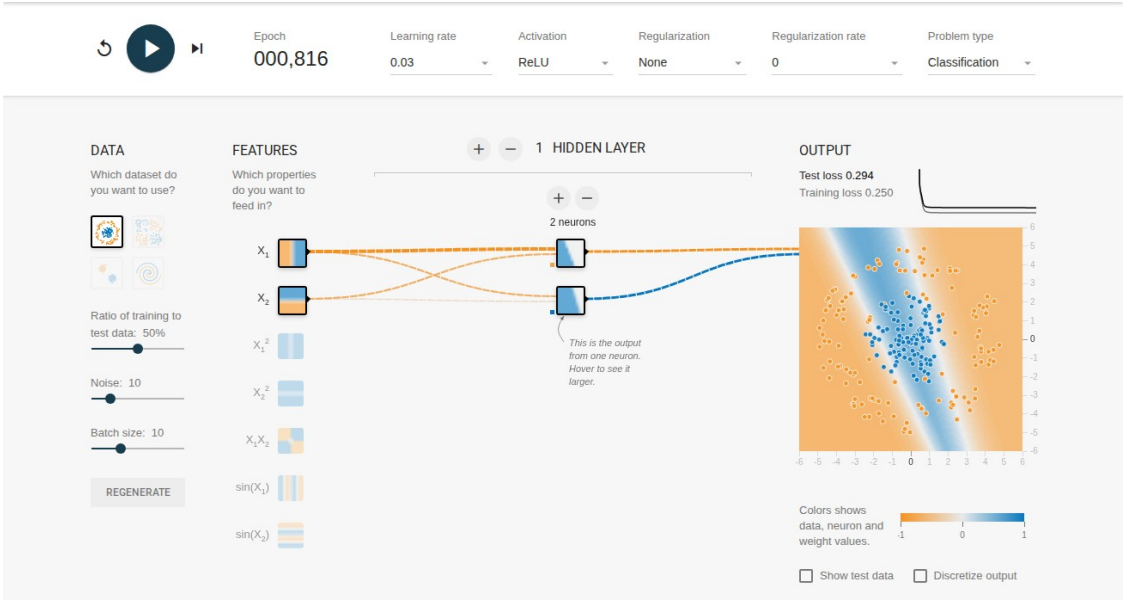
1 hidden neuron, ReLU: test loss 0.424, training loss 0.392 (epoch 000,521)

Will it learn nonlinearities? No. The output of a single neuron is computed as a single linear combination followed by an activation: a = f(w1·X1 + w2·X2 + b). Therefore, its decision boundary is constrained to a single hyperplane (a straight line in 2D), regardless of whether the activation is Linear or nonlinear. Concentric circular data cannot be separated by a single half-space, which is confirmed by the high test loss (≈0.42-0.51) and the diagonal linear boundary.

Task 2 – two hidden neurons, Linear vs ReLU



2 neurons, Linear: test loss 0.502, training loss 0.496 (epoch 000,620)

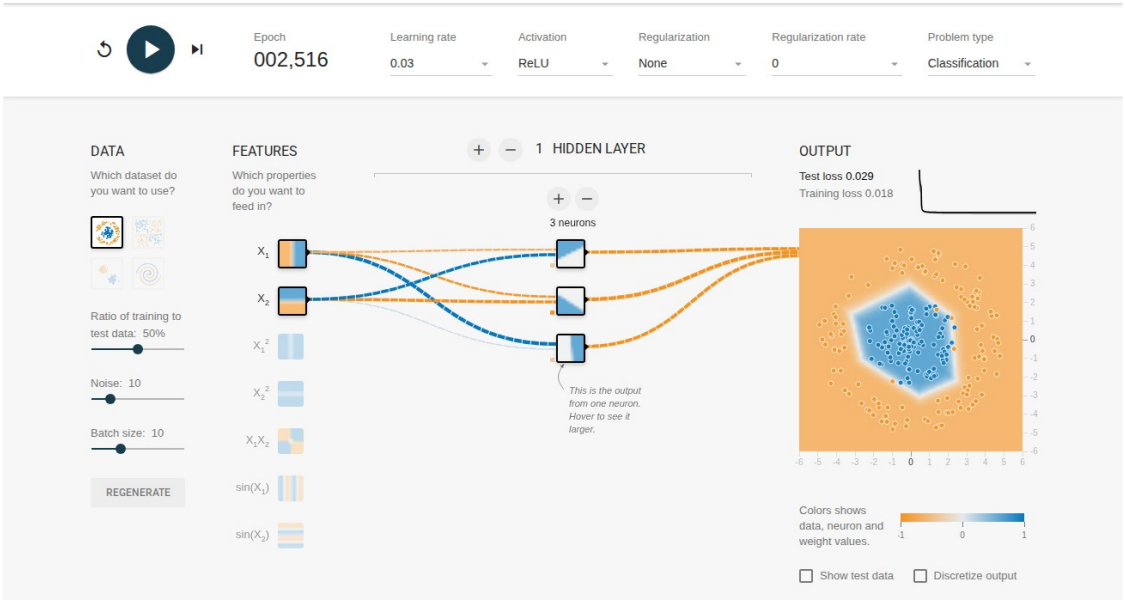


2 neurons, ReLU: test loss 0.294, training loss 0.250 (epoch 000,816)

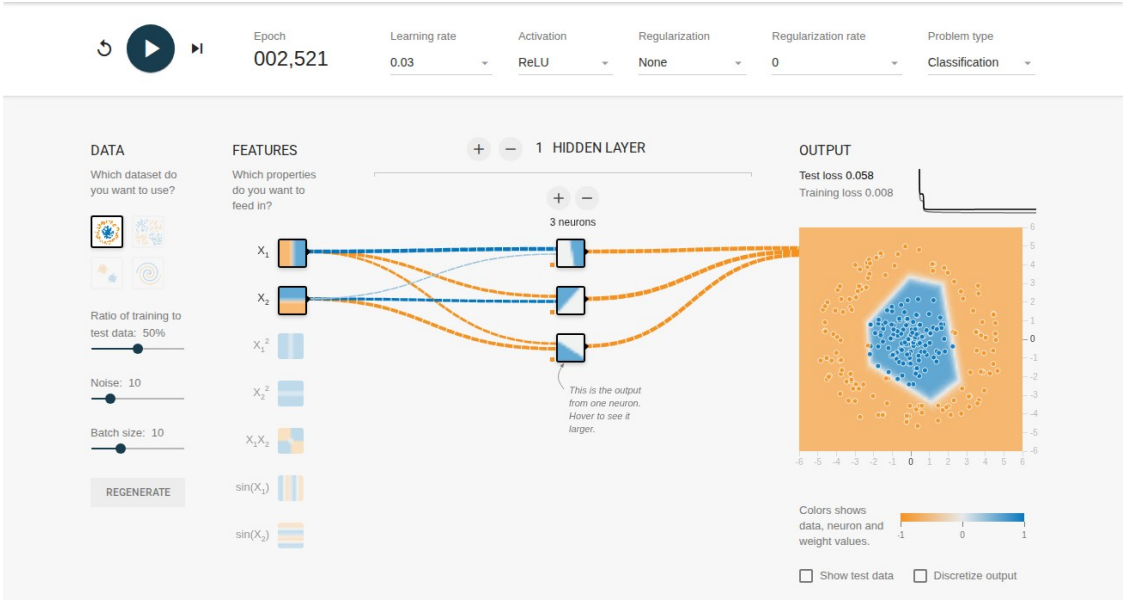
Can you create a model that can learn nonlinearities? With Linear activation, no. Any combination of linear layers simply collapses into another linear transformation (W2·(W1·X + b1) + b2 = W'·X + b'). The decision boundary remains strictly linear, and the loss stays at ~0.502. With ReLU, yes. The non-linear activation introduces piecewise linearity, allowing the network to combine two half-spaces into an open strip/band, dropping the test loss significantly to 0.294.

Is it capable to model the data effectively? Not effectively. Two linear boundaries can at most create an open slab between two lines. Because the boundary cannot close, orange points located at the top and bottom of the outer circle remain misclassified inside the strip, preventing the test loss from dropping below ~0.294.

Task 3 – three hidden neurons, ReLU



3 neurons, ReLU, run 1: test loss 0.029, training loss 0.018 (epoch 002,516)

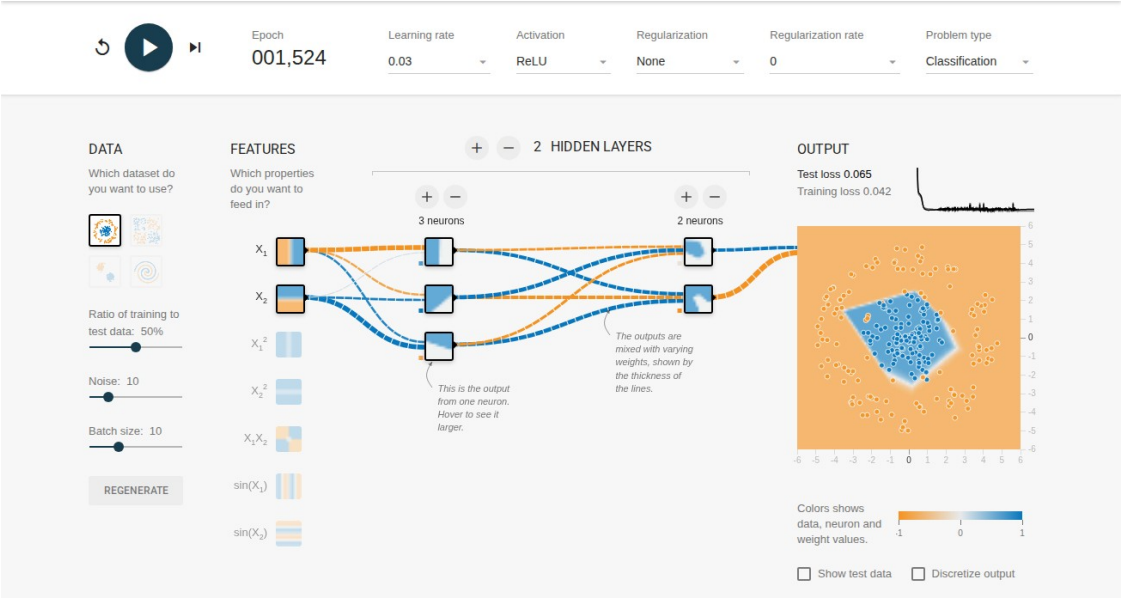


3 neurons, ReLU, run 2: test loss 0.058, training loss 0.008 (epoch 002,521)

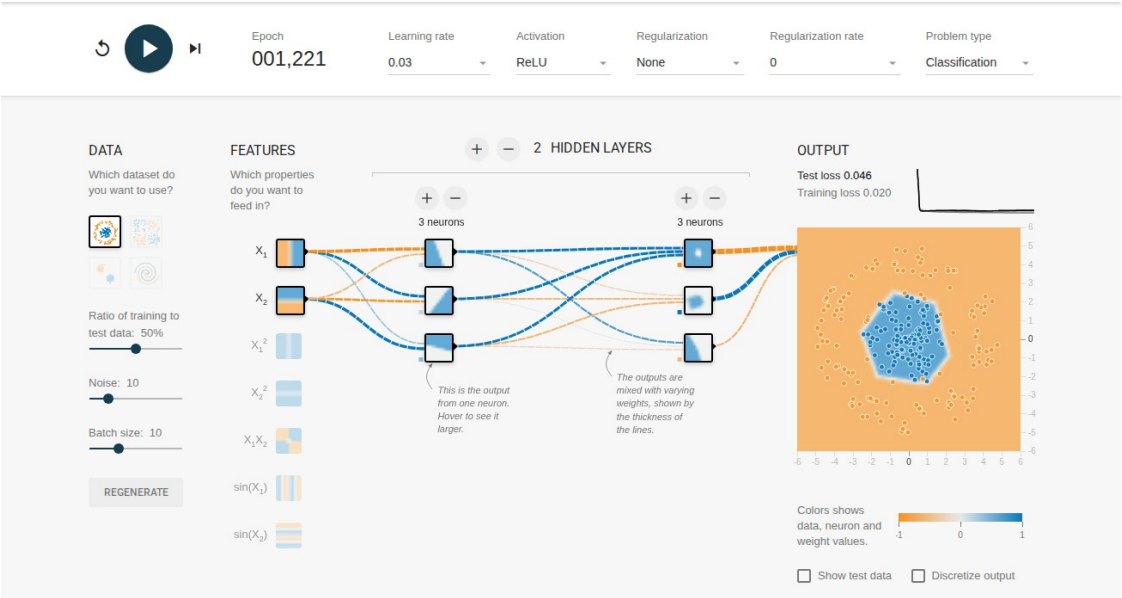
Is it capable to model the data effectively? Yes. Three ReLU neurons provide three intersecting hyperplanes, which is the exact mathematical minimum needed to form a closed convex polygon (a triangle) enclosing the inner circular cluster. The test loss drops down to 0.029-0.058.

How does the model quality vary from run to run? The quality varies depending on random weight initialization. In Run 1, the three lines converge symmetrically into a regular polygon around the blue points (test loss 0.029). In Run 2, an unfavorable initialization causes one boundary line to cut off an edge of the blue cluster before stabilizing, resulting in a slightly skewed boundary and a higher test loss (0.058). In both runs, training loss remains slightly lower than test loss due to 10% label noise.

Task 4 – model size and capacity



2 hidden layers (3, 2 neurons): test loss 0.065, training loss 0.042 (epoch 001,524)



2 hidden layers (3, 3 neurons): test loss 0.046, training loss 0.020 (epoch 001,221)

Smallest model achieving test loss ≤ 0.177: 1 hidden layer with 3 neurons (achieving test loss 0.029-0.058, well below the 0.177 threshold). A 2-neuron network plateaus at ~0.294, which fails to reach the target.

Impact of increasing model size:

Quality of fit: Adding layers and neurons produces a much smoother, rounded circular boundary rather than a sharp triangular enclosure. Test loss remains low (~0.046-0.065), confirming that extra capacity mainly improves geometric smoothness.

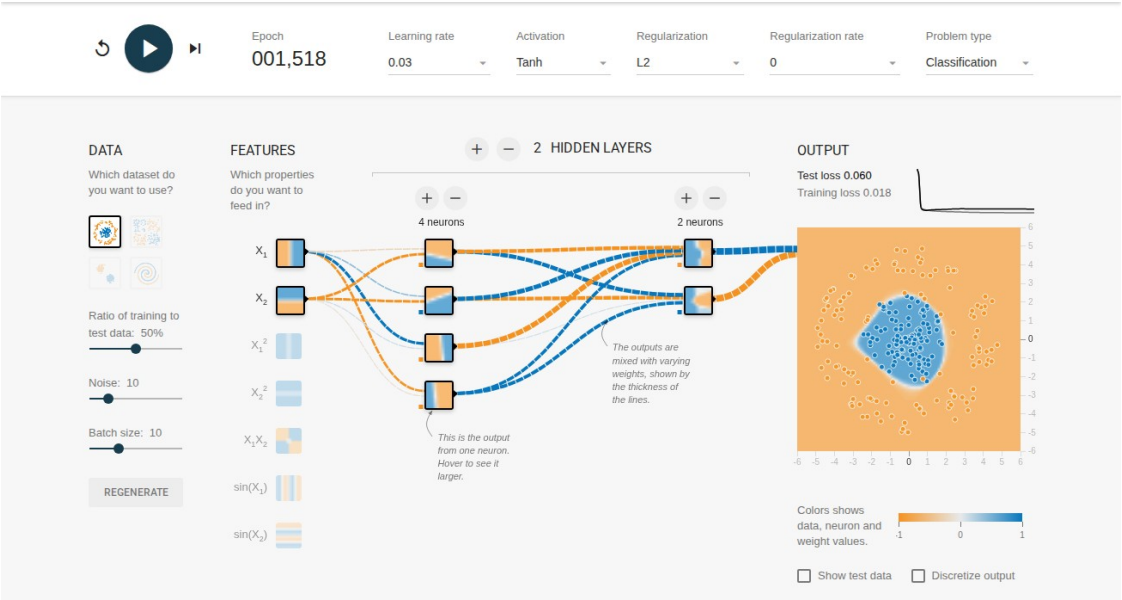
Convergence speed: Deeper networks converge in significantly fewer epochs (≈1200-1500 epochs vs ≈2500+ epochs for the 3-neuron network) because higher

capacity provides more gradient paths and reduces sensitivity to initial weights, though each epoch requires more floating-point operations.
Conclusion: While a minimal 3-neuron network is sufficient, scaling up slightly (e.g., 3, 2 or 3, 3) stabilizes optimization and eliminates run-to-run geometric variance, but excessive capacity risks fitting the 10% background noise.

Exercise 2 – Regularization

Settings: circle dataset, noise 10%, tanh activation, learning rate 0.03, network shape (4, 2), features X1 and X2 only.

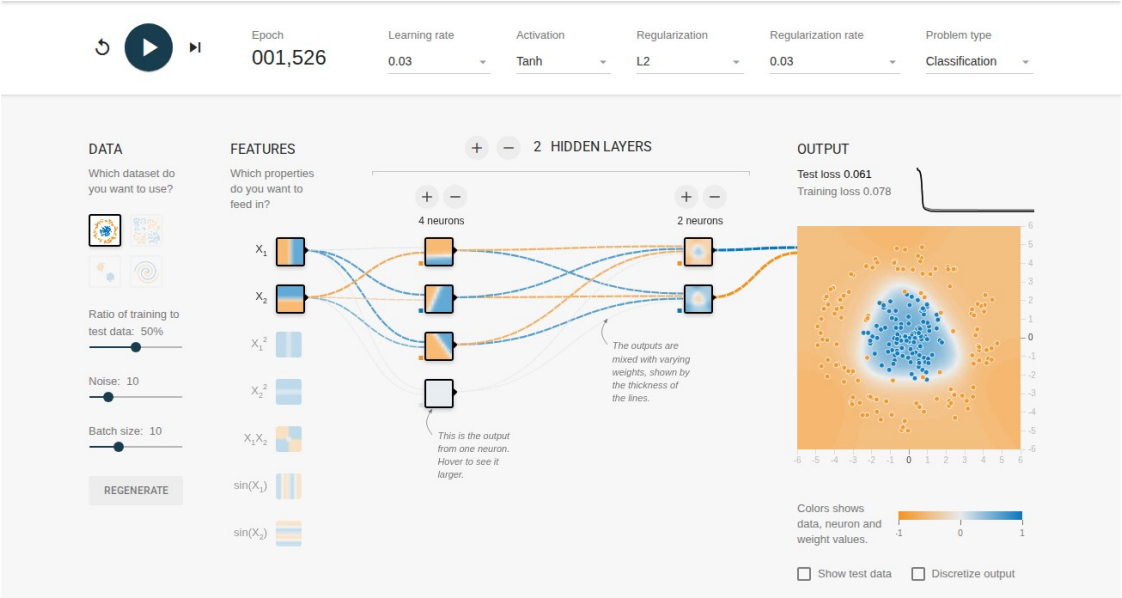
Task 1 – unregularized baseline (rate = 0)



Regularization rate 0 (L2): test loss 0.060, training loss 0.018 (epoch 001,518)

Losses: Test loss 0.060, training loss 0.018, resulting in a noticeable generalization gap of 0.042.
Learned weights: Several thick weight connections appear between input features and the first layer, as well as into the output neuron, showing that the network assigns high weights to fit noisy training points.
Decision boundary: While the general circle is captured, the boundary displays localized warping and faint streaks in the orange region, reflecting mild overfitting to the 10% noise.

Task 2 – regularized model (rate = 0.03)



Regularization rate 0.03 (L2): test loss 0.061, training loss 0.078 (epoch 001,526)

How does test loss differ from Task 1? The test loss decreases slightly or remains competitive (0.061 vs 0.060), while avoiding overfitting to noise.

How does the test-training difference differ? The generalization gap narrows from 0.042 in Task 1 down to -0.017. Regularization penalizes extreme weights, preventing the model from fitting spurious noise in the training set.

How do learned weights differ? With L2 regularization enabled, the network connections are noticeably thinner. Weight decay shrinks the magnitude of weights toward zero, producing a more constrained hypothesis class.

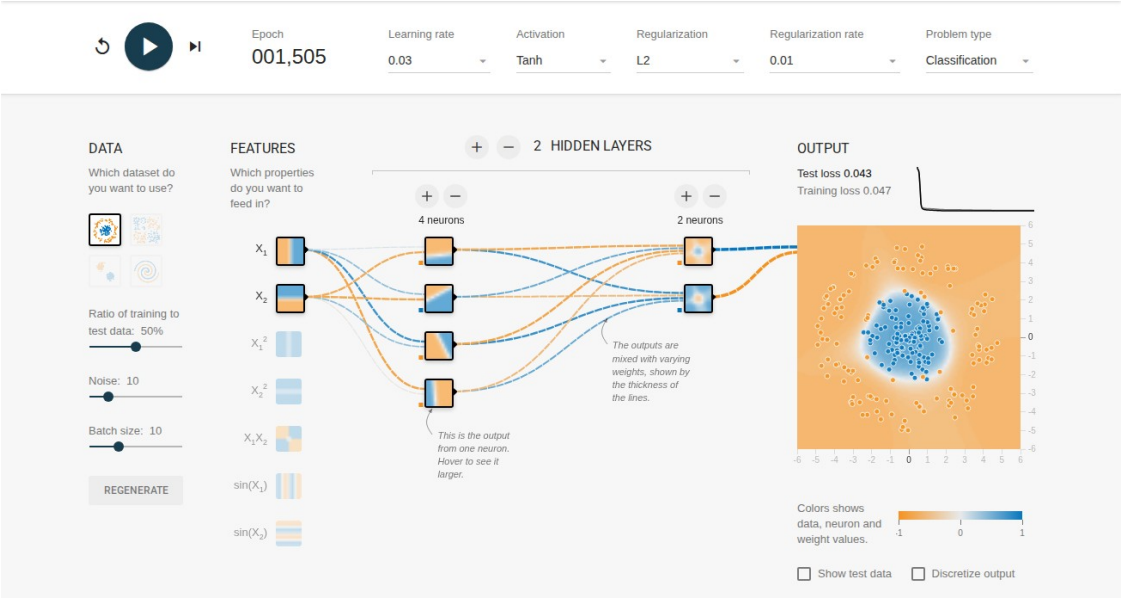
What do your results say about model complexity? The (4, 2) network has sufficient parameters to overfit the 10% label noise when unconstrained. L2 regularization acts as an effective complexity control: it penalizes large weight norms, smooths the output decision surface, reduces the gap between training and test loss, and ensures robustness against noisy observations.

Task 3 – regularization parameter exploration

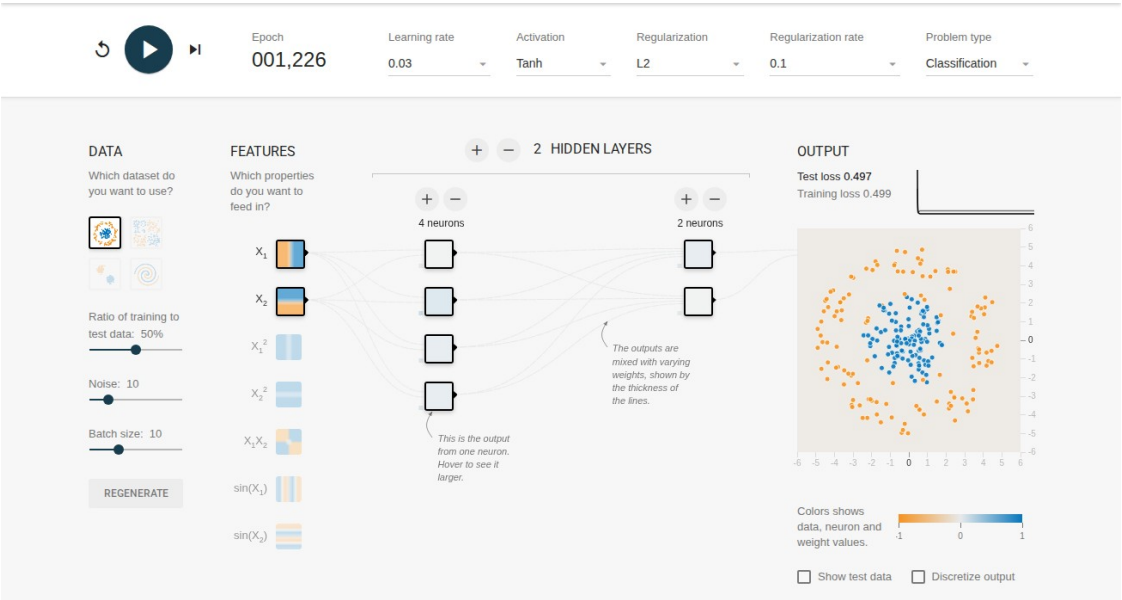
To evaluate how the regularization strength impacts network behavior, we trained the identical architecture across multiple L2 rates for at least 1200 epochs:

Rate	Test loss	Training loss	Test − training gap
0	0.060	0.018	0.042
0.001	0.058	0.023	0.035
0.003	0.054	0.029	0.025

0.01	0.043	0.047	-0.004
0.03	0.061	0.078	-0.017
0.1	0.497	0.499	-0.002



Rate 0.01 (L2): test loss 0.043, training loss 0.047 (epoch 001,505)



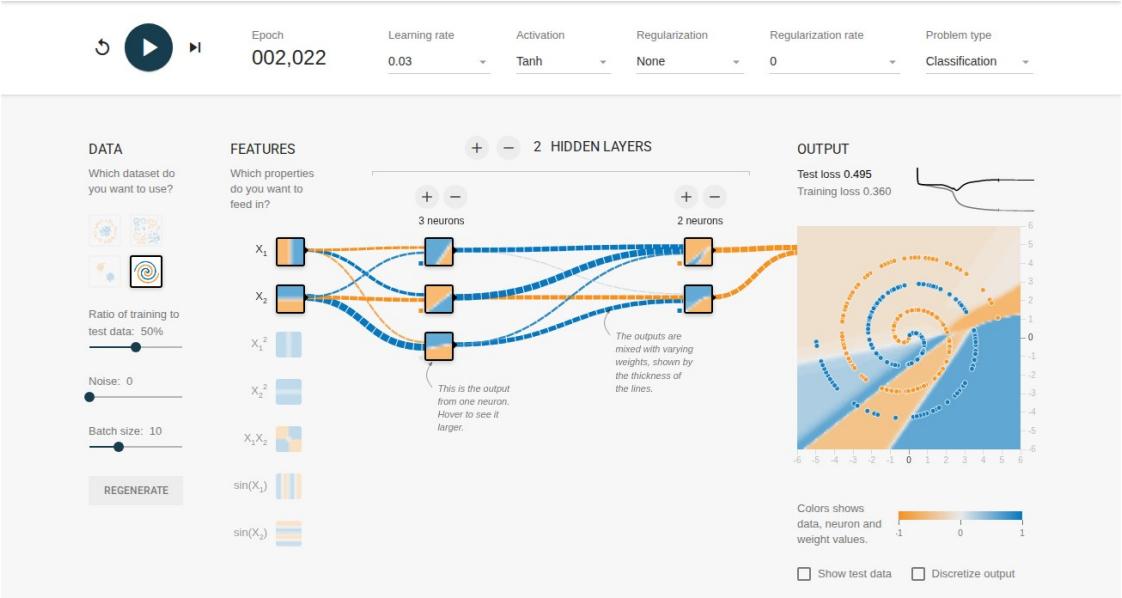
Rate 0.1 (L2): test loss 0.497, training loss 0.499 (epoch 001,226)

Performance Analysis: A moderate regularization rate of 0.01 to 0.03 achieves the best trade-off. At rate 0 (no penalty), the model slightly overfits with larger weights and an uneven boundary. At small rates (0.001 - 0.01), weights are regularized without impairing capacity. At rate 0.03, the gap is minimized while preserving high accuracy. At rate 0.1, regularization becomes too aggressive (underfitting), causing the training and test loss to rise as the weights are over-constrained.

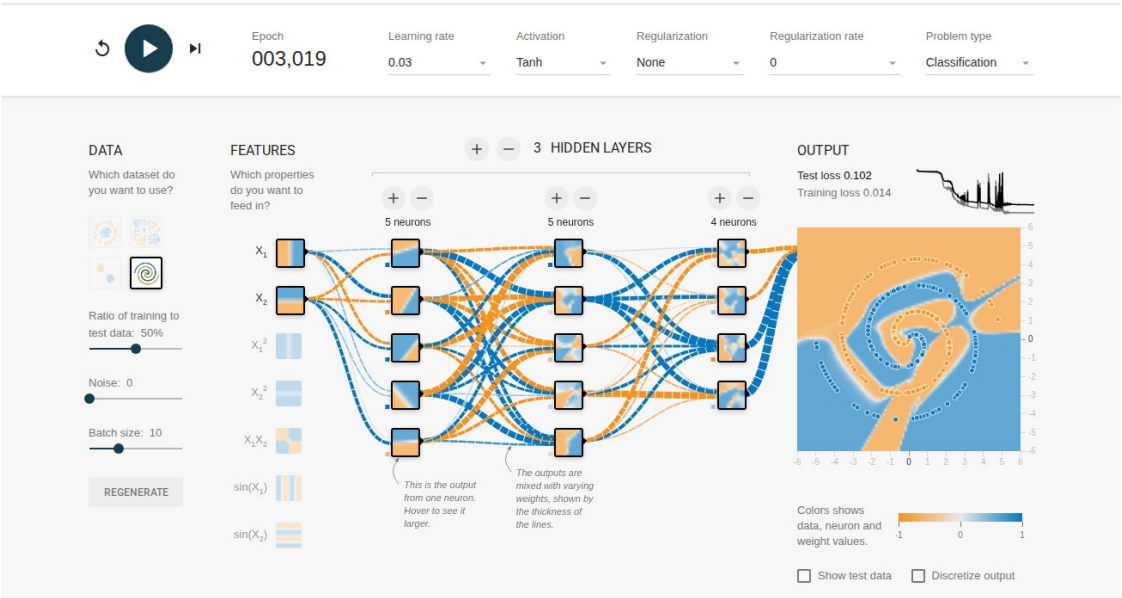
Exercise 3 – Spiral dataset

Dataset: spiral, noise: 0%, tanh activation, learning rate 0.03.

Task 1 – using X1 and X2 features only



2 hidden layers (3, 2 neurons): test loss 0.495, training loss 0.360 (epoch 002,022)



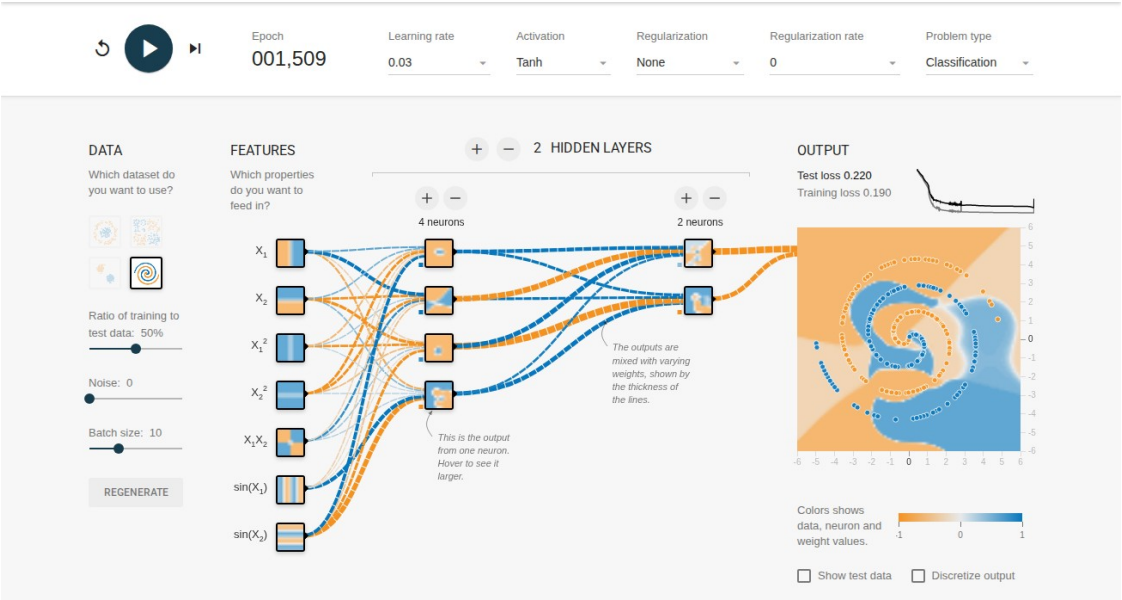
Best deep model, 3 hidden layers (5, 5, 4 neurons): test loss 0.102, training loss 0.014 (epoch 003,019)

What is the best test loss you can get? With raw features X1 and X2 only, the best test loss reached was 0.102 using a deep 3-layer architecture (5, 5, 4 neurons). In contrast, the smaller (3, 2) network completely fails, unable to represent the winding topology and plateauing at a test loss of ~0.495.

How smooth is the model output surface? The output surface is irregular, segmented, and not smooth. Because the network must construct the highly nonlinear, intertwined spiral arms solely by composing linear tanh boundaries, the decision region displays sharp angular corners, disconnected artifacts, and vertical streaks. The loss curve exhibits

oscillations, indicating that optimization is fragile and struggles to fit the intricate curvature.

Task 2 – feature engineering (cross-products and trigonometric features)



Engineered features (X1, X2, X1X2, sin(X1), sin(X2), $X1^2$, $X2^2$): test loss 0.220, training loss 0.190 (epoch 001,509)

Did you get a better model? Yes, significantly better. By feeding domain-appropriate features such as sin(X1), sin(X2), X1·X2, and squared terms, the network learns the spiral arms with a smaller 2-layer network (4, 2 neurons) in only 1500 epochs, reaching a test loss of 0.220 without needing deep architectures.

Is the model output surface smoother? Yes, dramatically smoother. The sinusoidal input features naturally provide periodic and circular geometric priors, allowing the network to wrap around the spiral arms with continuous, curved contours rather than jagged piecewise linear approximations. The boundary aligns cleanly with the spiral manifold with zero artifacts.